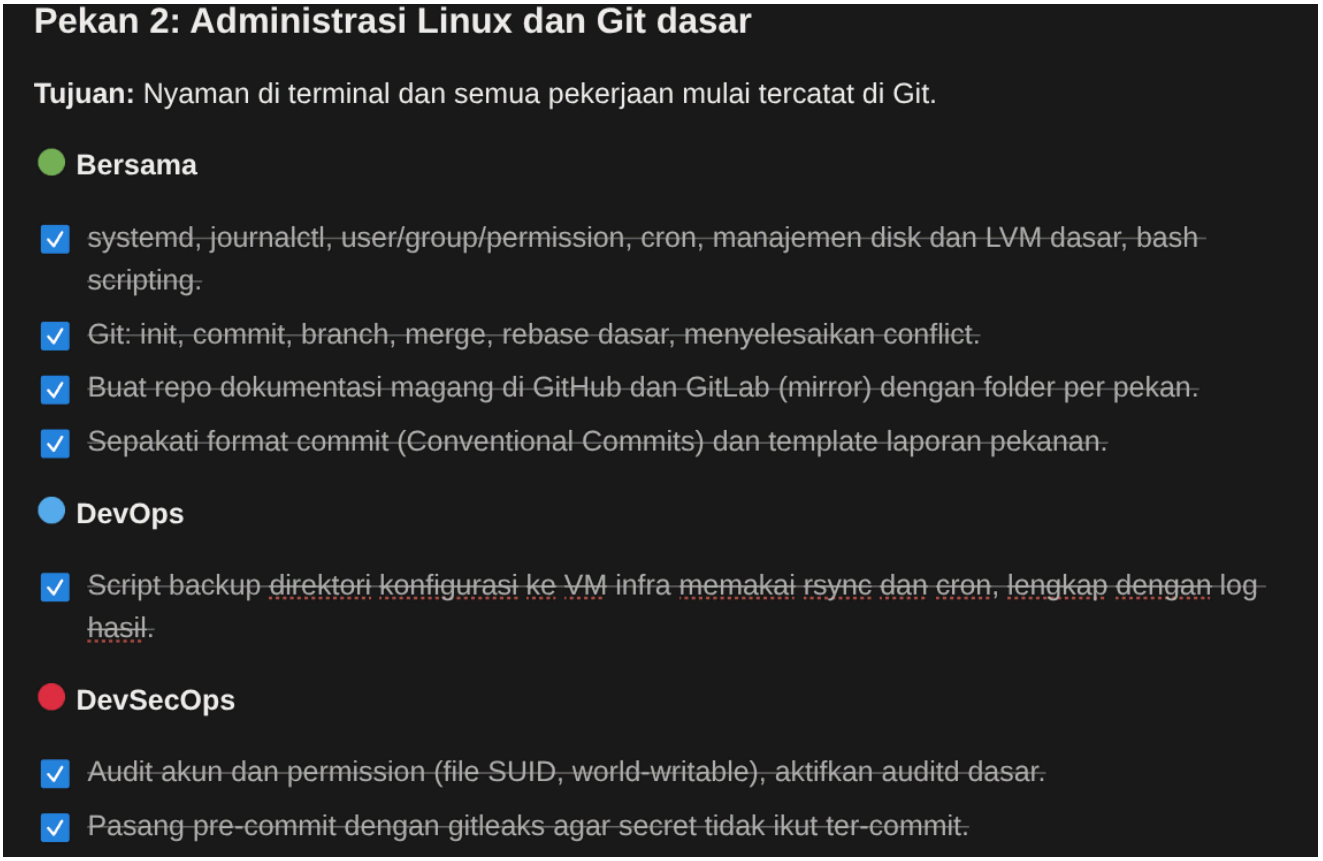
Dokumentasi Pekan 2

hai, kali ini kita akan membuat dokumentasi untuk pekan 2 yang mana berisi poin poin pada gambar berikut:

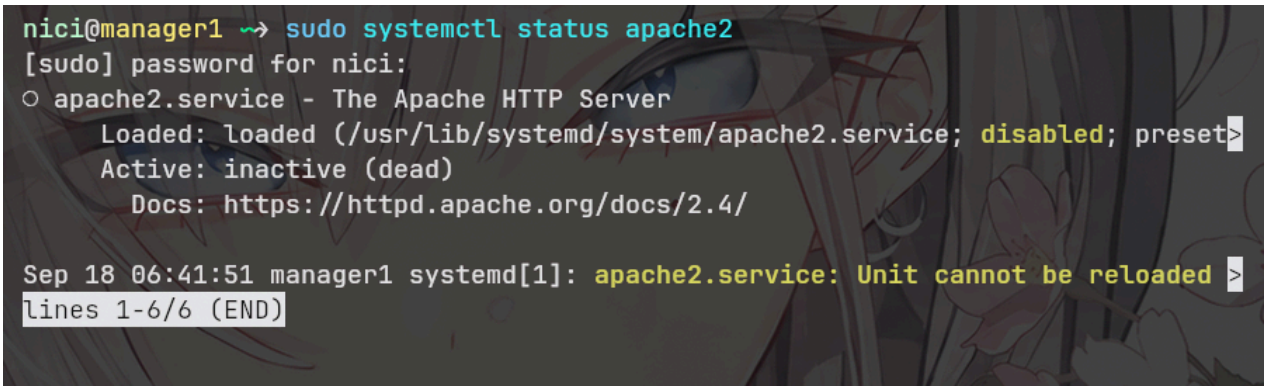


nah kita akan mulai saja dari systemd,journalctl,group,permission,manajemen disk, lvm, dan bash scripting yang mana ini hanya akan full dengan penjelasan aja yang aku mulai aja dari systemd, nah apa itu systemd? intinya systemd itu adalah sebuat init system yang aku pribadi sangat tidak menyukai dan sebenarnya ogah menggunakannya, init system sendiri adalah sebuah program yang akan dijalankan pertama setelah komputer/laptop dinyalakan yang mana init system itu menjalankan program program inti os yang berjalan di latar belakang seperti program untuk network, journaling dll yang masalahnya systemd ini sok asik dengan ikut mengurusi proses proses yang seharusnya tidak dijalankan oleh init, kalau ingin tau berbagai hal yang dilakukan oleh systemd, bisa dilihat di halaman https://suckless.org/sucks/systemd/, nah balik lagi ke topik, intinya di systemd itu kita bisa menambahkan program untuk dijalankan di latar belakang(daemon) ataupun menghapusnya, menambahkan program ke list program yang akan dinyalakan oleh systemd maupun menghapusnya, dan masih banyak lagi bahkan sampai hal yang gapenting seperti dns pun juga dia ikut ikutan ngatur.

Systemd

untuk list perintah yang bisa digunakan seperti berikut:

1. sudo systemctl start (nama service) - ini buat menjalankan sesuatu di latar belakang tapi tidak secara permanen jadi kalau kita reboot device kita ya kita harus nyalain manual
2. sudo systemctl stop (nama service) - kebalikannya start
3. sudo systemctl status (nama service) - ini untuk mengecek status dari program yang berjalan di latar belakang, contohnya adalah



4. sudo systemctl restart (nama service) - intinya buat restart suatu service, kalau ada start dan stop ya pasti ada restart
5. sudo systemctl enable (nama service) - nah ini buat bikin service yang dijalankan di latar belakang itu tercatat permanen(selama ga dihapus dan ssd diformat ulang) di latar belakang(daemon)
6. sudo systemctl disable (nama service) - kebalikannya enable
7. sudo systemctl reload (nama service) - kalau restart itu kan literally mati dulu baru dihidupin lagi, kalau reload ya sama tapi tidak mati dulu service nya jadi hanya file konfigurasi aja yang terupdate

yah itu aja sih kalau perintah systemctl yang cukup untuk dipahami dan selanjutnya adalah journalctl, nah journalctl ini juga sebenarnya cukup kupertanyakan bukan karena fungsinya tapi journalctl dijalankan di latar belakang(daemon) yang menurutku harusnya kalau journaling ya manual karena ga setiap saat aku perlu journaling tapi balik lagi journalctl itu software journaling yang mana journaling itu gampangnya kaya kita bikin copy an dari suatu informasi, tapi bukan backup ya, tapi intinya kurang lebih sama fungsinya walaupun tidak secara harfiah sama

journalctl

nah journalctl sendiri juga punya beberapa command yang mana terdiri dari:

1. journalctl - buat liat semua log, kalo ditambah log -e itu buat liat yang terakhir dan -r itu kebalikan aja urutannya berarti yang baru diatas
2. journalctl -U (nama service) - buat liat log dari service yang mau dibaca lognya
3. journalctl –since - ini buat filter waktu, contoh kalo mau sejam yang lalu ya journalctl –since “1 hour ago” yang mana akan diperlihatkan log sejam yang lalu
4. journalctl -p - ini buat liat yang priority
5. journalctl -b - ini buat liat log dari boot nya
6. journalctl -k - cuma log kernel (dmesg-style)

7. journalctl --disk-usage - cek berapa besar journal makan disk
8. journalctl --vacuum-time=7d - hapus log lebih lama dari 7 hari

Git / Version Control System

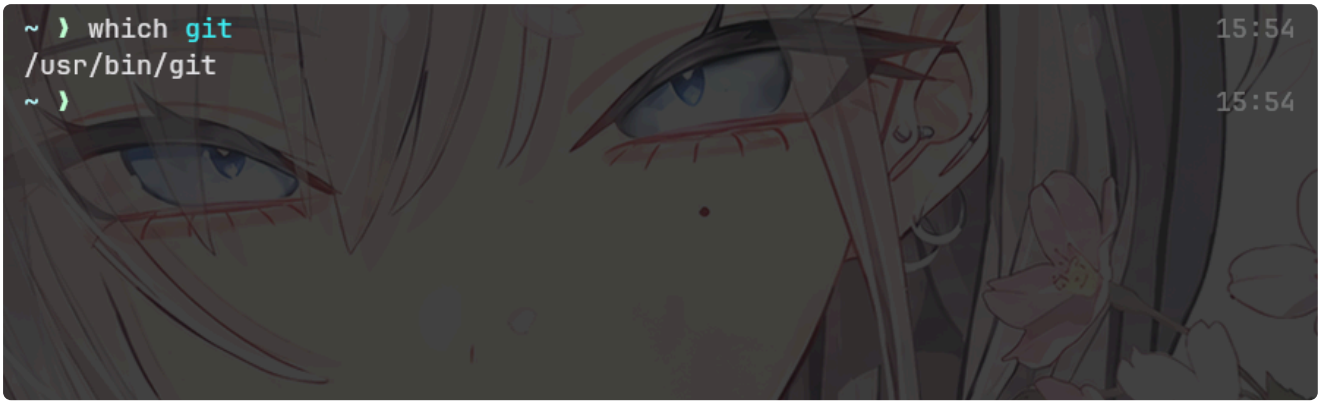
lalu next kita akan nyemplung di dunia vcs(version control system) yang mana tools yang paling josjis untuk digunakan adalah git, git sendiri adalah sebuah tools vcs yang dipake buat untuk melacak dan mencatat semua perubahan yang ada di suatu repository, dah kita akan bikin repository github aja, pertama langsung aja kita instal git dengan command:

```
sudo apt install git
```

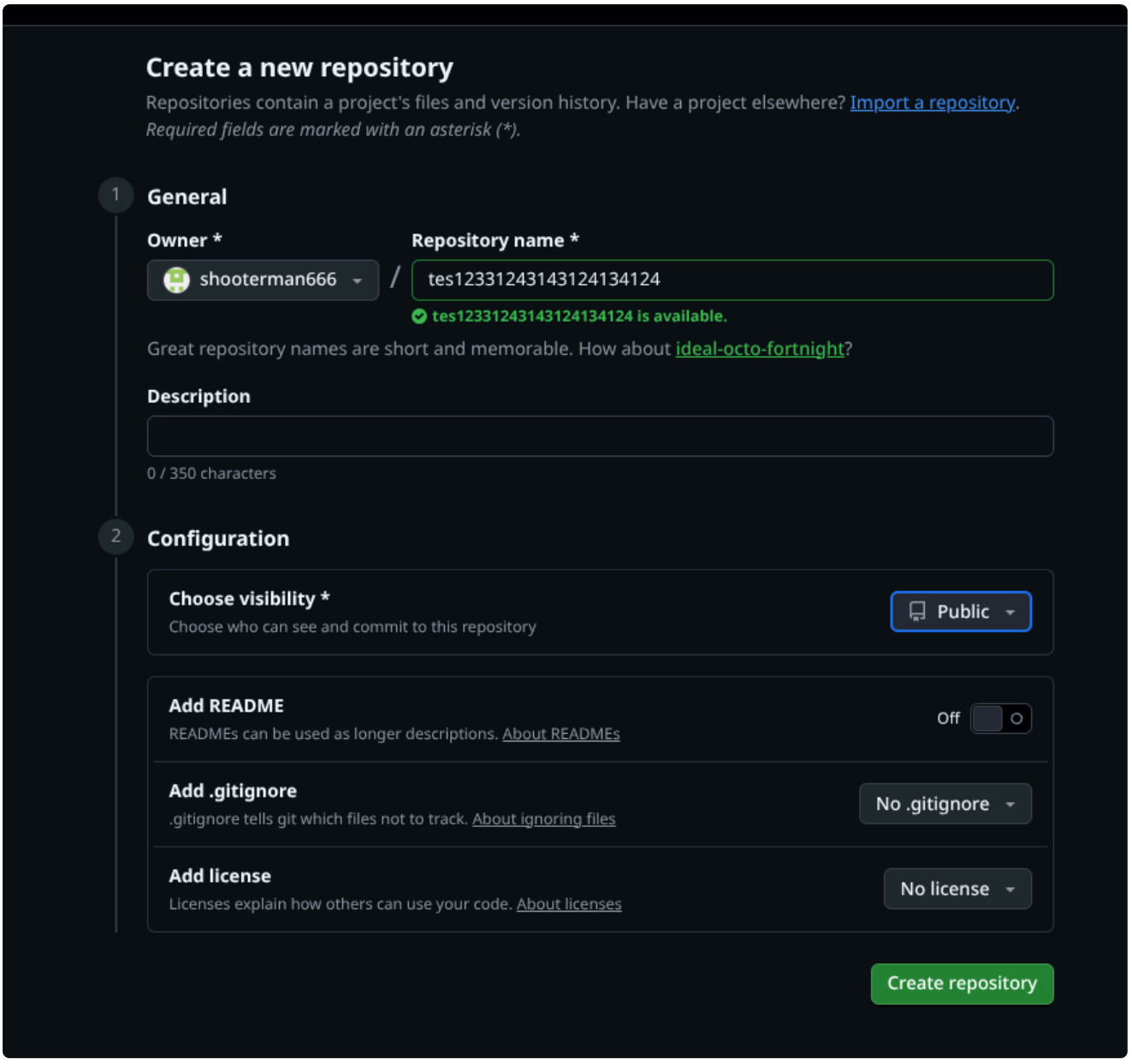
setelah itu kita validasi git nya dengan command

```
which git
```

yang mengeluarkan output



setelah itu kita coba bikin repo di github dulu dengan membuka web https://github.com dan silahkan login terus klik new repository lalu tinggal disesuaikan aja repo nya mau gimana,

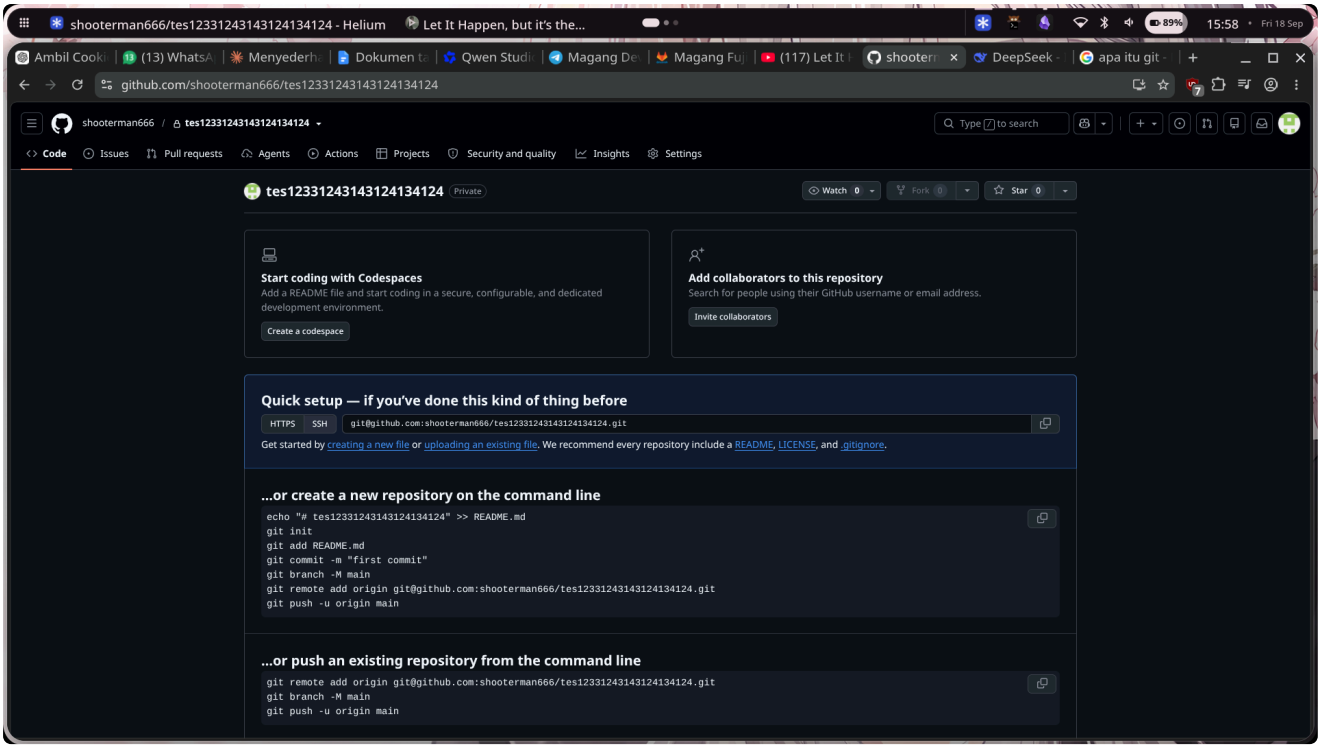


pada gambar tersebut, kita bisa bikin repo nya itu public atau private, bisa menambahkan readme, gitignore, maupun lisensi kalau kita bikin software wajib ada lisensinya

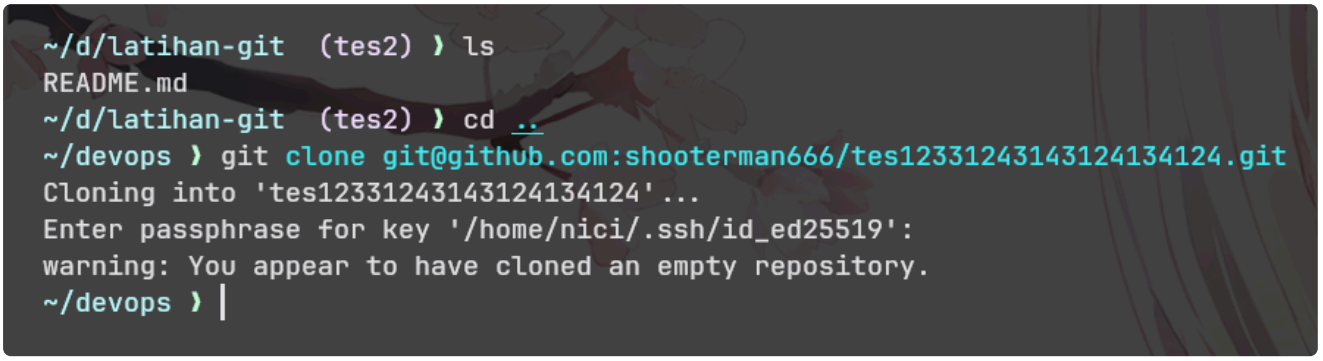
setelah terbuat maka akan ada tampilan seperti ini,

```
~/d/latihan-git  (tes2) ❱ ls
README.md
~/d/latihan-git  (tes2) ❱ cd ..
~/devops ❱ git clone git@github.com:shooterman666/tes12331243143124134124.git
Cloning into 'tes12331243143124134124'...
Enter passphrase for key '/home/nici/.ssh/id_ed25519':
warning: You appear to have cloned an empty repository.
~/devops ❱ cd tes12331243143124134124/
~/d/tes12331243143124134124  (main) ❱
```

dan ki



nah di bagian quick setup, kita bisa melakukan cloning, yang intinya kita bisa clone seluruh isi dari repository nya ke direktori kita sendiri dan modifikasi sesuka kita, tapi disini kita praktek buat repo yang mana aku akan ngeclone itu,



nah itu kira kira hasilnya yang mana karena repo nya kubuat private ya aku perlu isi kredensial dan disitu kebetulan aku pakai ssh key, bisa juga pake https kok dan hasilnya juga sama dan gausa pedulikan warningnya dan kita akan coba cek apakah ada direktori yang ke clone, dan kita lihat udah ada dan kita bisa upload kode atau file apapun ke github, dan kita akan coba menambahkan file sembarang deh,



nah kita liat itu dan diatas aku membuat file p.txt dan setelah itu aku ketik

```
git add .
```

yang berarti menambahkan semua file yang dibuat dan bisa menggunakan

```
git add (nama file)
```

untuk menambahkan file tertentu dan kita pakai

```
git status
```

untuk melihat status dari repo kita dan kita juga perlu commit dengan command

```
git commit -m "pesan apapun"
```

untuk menambahkan commit kita karena kalau tidak, kita tidak akan bisa push karena setiap perubahan wajib menggunakan commit agar segala perubahan bisa ter record lalu ada juga yang namanya branch, intinya kita bisa membuat jalur development selain main karena main adalah branch utama dan saat di build maka main yang akan ter build, jadi kita bisa membuat branch baru dengan command

```
git checkout -b (nama branch)
```

dan dilihat di bagian kanan dari nama direktori yang mana main berganti ke tes1 dan jika kita ingin kembali ke branch main bisa dengan command

```
git checkout main
```

dan kalau ingin push, kita tinggal push dengan command

```
git push origin main
```

untuk ngepush branch mana yang akan kita push jadi kalau kita ngepush maka kita akan ngepush perubahan pada branch main

Format Commit

udah sekian soal git jadi kita akan next ke format commit yang mana jujur soal format commit aku pribadi juga bingung jadi aku berdiskusi dengan beberapa orang dan kebentuk format baru yang mana contohnya adalah

```
<tipe commit>: (deskripsi singkat)
```

nah untuk tipe tipe nya sendiri itu ada,

feat - nambah fitur/temuan baru
fix - perbaikan (misal salahin di report sebelumnya)
docs - perubahan dokumentasi/report doang, nggak ada kode
refactor - rapiin struktur tanpa ubah isi
chore - hal remeh (rename file, update .gitignore, dll)

jadi kalau kita ambil contoh commit nya berarti,

```
git commit -m "feat: add initial recon notes for auth module"
```

Backup Konfigurasi VM Infra

nah dan kita akan lanjut ke bikin skrip backup yang mana tujuan backup kita ini adalah vm infra, yang mana untuk skripnya aku coba backup beberapa konfigurasi seperti net ssh, fail2ban, ufw, netplan dll lalu kita pindahkan ke vm infra menggunakan skrip yang mana akan menggunakan ssh keygen biar simpel, dari command yang "ssh copy to " ke cuma ssh lalu tembak alias aja, contohnya disini aku bikin ip 192.168.56.13 jadi alias "infa" aja dengan cara pada dokumentasi sebelumnya yang mana kita generate ssh key baru lalu diberikan ke vm tujuan tapi ini kita akan pakai sudo karena direktori yang kita akan pindahkan itu butuh kredensia root, kita gabisa pindahkan /etc/netplan/netpan.conf ke vm lain tanpa akses root jadi command yang akan kita pake adalah(btw ini dijalankan di manager, worker1 dan 2 karena host disini cuma untuk ssh aja)

```
sudo ssh-keygen -t ed25519 -f /root/.ssh/id_backup -N "" -C "berikan pesan kalau perlu kalo ga ya gapapa"
```

lalu setelah key ter generate kita jalankan command

```
sudo cat /root/.ssh/id_backup.pub
```

untuk cek key yang terbuat di direktori kita lalu kita jalankan command ini pada vm infra(kalau mau copy dari manager langsung ke infra juga bisa, gausah ngetik manual kaya gini):

```
echo "ini public key yang sebelumnya keluar paste ke sini" >> ~/.ssh/authorized_keys
```

lalu kita balik lagi ke manager dan setelah itu kita edit konfigurasi ssh nya pada /root/.ssh/config dengan isian seperti ini:

```
Host infra
    HostName 192.168.56.13
    User nici
    IdentityFile /root/.ssh/id_backup
    IdentitiesOnly yes
```

host itu kita mau kasi dia alias apa
hostname tuh kita bisa isi dengan ip dari vm infra
identify file itu kita masukkan lokasi dimana generated key nya dibuat
identitiesonly sih kita nyuruh dia bener bener pake satu file spesifik yang udah kita set sebelumnya jadi file itu adalah file yang benar benar terpakai, bukan pake file lain (kalo ada pubkey lain di vm kan siapa tau)

lalu jangan lupa juga permission nya, kita tetap di vm manager dan kita juga wajib berikan permission karena ssh itu ribet banget masalah beginian, jadi kita jalankan command ini:

```
sudo chmod 700 /root/.ssh
sudo chmod 600 /root/.ssh/config /root/.ssh/id_backup
```

seperti ini hasilnya



disitu juga ada command tes ssh juga dan berhasil

dan juga ini kita berikan juga permission di vm infra nya dengan command chmod yang sama seperti di manager

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

nah setelah itu kita tinggal buat skripnya dan yang mana ini skrip aku mintol ai aja buat bikinnya apalagi kalau direktori segitu yang di backup masi cukup simpel skripnya

kira kira isian skripnya gini,

```
#!/bin/bash
################################################################################
```

```
###
# backup-config.sh — Backup direktori konfigurasi ke VM infra via rsync
# Didesain buat dijalankan manual ATAU lewat cron di
manager1/worker1/worker2
################################################################################
###

set -uo pipefail

# ==================== KONFIGURASI ====================

# Direktori konfigurasi yang mau di-backup (sesuaikan sesuai VM)
BACKUP_DIRS=(
    "/etc/ssh"
    "/etc/ufw"
    "/etc/fail2ban"
    "/etc/netplan"
    "/etc/hosts"
    "/etc/hostname"
)

# Tujuan backup: VM infra
REMOTE_USER="nici"
REMOTE_HOST="infra"
REMOTE_BASE_DIR="/home/${REMOTE_USER}/backups/$(hostname)"

# Lokasi log lokal (di VM yang jalanin backup, bukan di infra)
LOG_DIR="/var/log/backup-config"
LOG_FILE="${LOG_DIR}/backup-$(date +%Y%m%d-%H%M%S).log"
LOG_RETENTION_DAYS=30

# ==================== HELPER ====================

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m';
BLUE='\033[0;34m'; NC='\033[0m'

log() {
    local level="$1"; shift
    local msg="$*"
    local ts
    ts="$(date '+%Y-%m-%d %H:%M:%S')"
    echo "[${ts}] [${level}] ${msg}" >> "$LOG_FILE"

    case "$level" in
        INFO)    echo -e "${BLUE}[INFO]${NC} ${msg}" ;;
        SUCCESS) echo -e "${GREEN}[SUCCESS]${NC} ${msg}" ;;
        WARNING) echo -e "${YELLOW}[WARNING]${NC} ${msg}" ;;
        ERROR)   echo -e "${RED}[ERROR]${NC} ${msg}" ;;
    esac
}
```

```
# ==================== PRA-SYARAT ====================

check_prerequisites() {
    command -v rsync &> /dev/null || { log ERROR "rsync tidak ditemukan.
Install dengan: sudo apt install rsync"; exit 1; }

    mkdir -p "$LOG_DIR"

    log INFO "Cek koneksi SSH ke ${REMOTE_HOST}..."
    if ! ssh -o BatchMode=yes -o ConnectTimeout=5
"${REMOTE_USER}@${REMOTE_HOST}" "exit" 2>> "$LOG_FILE"; then
        log ERROR "Gagal konek ke ${REMOTE_USER}@${REMOTE_HOST} via SSH
key. Pastikan key sudah di-copy dan tidak pakai passphrase (buat cron
unattended)."
        exit 1
    fi
    log SUCCESS "Koneksi SSH ke ${REMOTE_HOST} OK."

    ssh "${REMOTE_USER}@${REMOTE_HOST}" "mkdir -p ${REMOTE_BASE_DIR}" 2>>
"$LOG_FILE"
}

# ==================== BACKUP ====================

run_backup() {
    local total=0
    local success=0
    local failed=0
    local start_time
    start_time=$(date +%s)

    log INFO "=== Mulai backup dari $(hostname) ke
${REMOTE_HOST}:${REMOTE_BASE_DIR} ==="

    for dir in "${BACKUP_DIRS[@]}"; do
        ((total++))

        if [ ! -e "$dir" ]; then
            log WARNING "Skip: ${dir} tidak ditemukan di sistem ini."
            continue
        fi

        log INFO "Backup ${dir}..."

        # -a  : archive mode (preserve permission, owner, timestamp,
symlink, dll)
        # -z  : compress saat transfer
        # -R  : preserve relative path (biar struktur /etc/ssh dst
konsisten di sisi infra)
```

```
        # --delete : hapus file di tujuan yang sudah tidak ada di source
(biar backup selalu mirror)
        if rsync -azR --delete "$dir"
"${REMOTE_USER}@${REMOTE_HOST}:${REMOTE_BASE_DIR}/" >> "$LOG_FILE" 2>&1;
then
            log SUCCESS "Backup ${dir} berhasil."
            ((success++))
        else
            log ERROR "Backup ${dir} GAGAL. Cek detail di ${LOG_FILE}."
            ((failed++))
        fi
    done

    local end_time duration
    end_time=$(date +%s)
    duration=$((end_time - start_time))

    log INFO "=== Ringkasan: ${success}/${total} berhasil, ${failed}
gagal, durasi ${duration}s ==="

    if [ "$failed" -gt 0 ]; then
        log ERROR "Backup selesai DENGAN error. Cek ${LOG_FILE}."
        return 1
    else
        log SUCCESS "Semua backup berhasil tanpa error."
        return 0
    fi
}

# ==================== CLEANUP LOG LAMA ====================

cleanup_old_logs() {
    find "$LOG_DIR" -name "backup-*.log" -mtime "+${LOG_RETENTION_DAYS}" -
delete 2>/dev/null
    log INFO "Log lebih lama dari ${LOG_RETENTION_DAYS} hari sudah
dibersihkan."
}

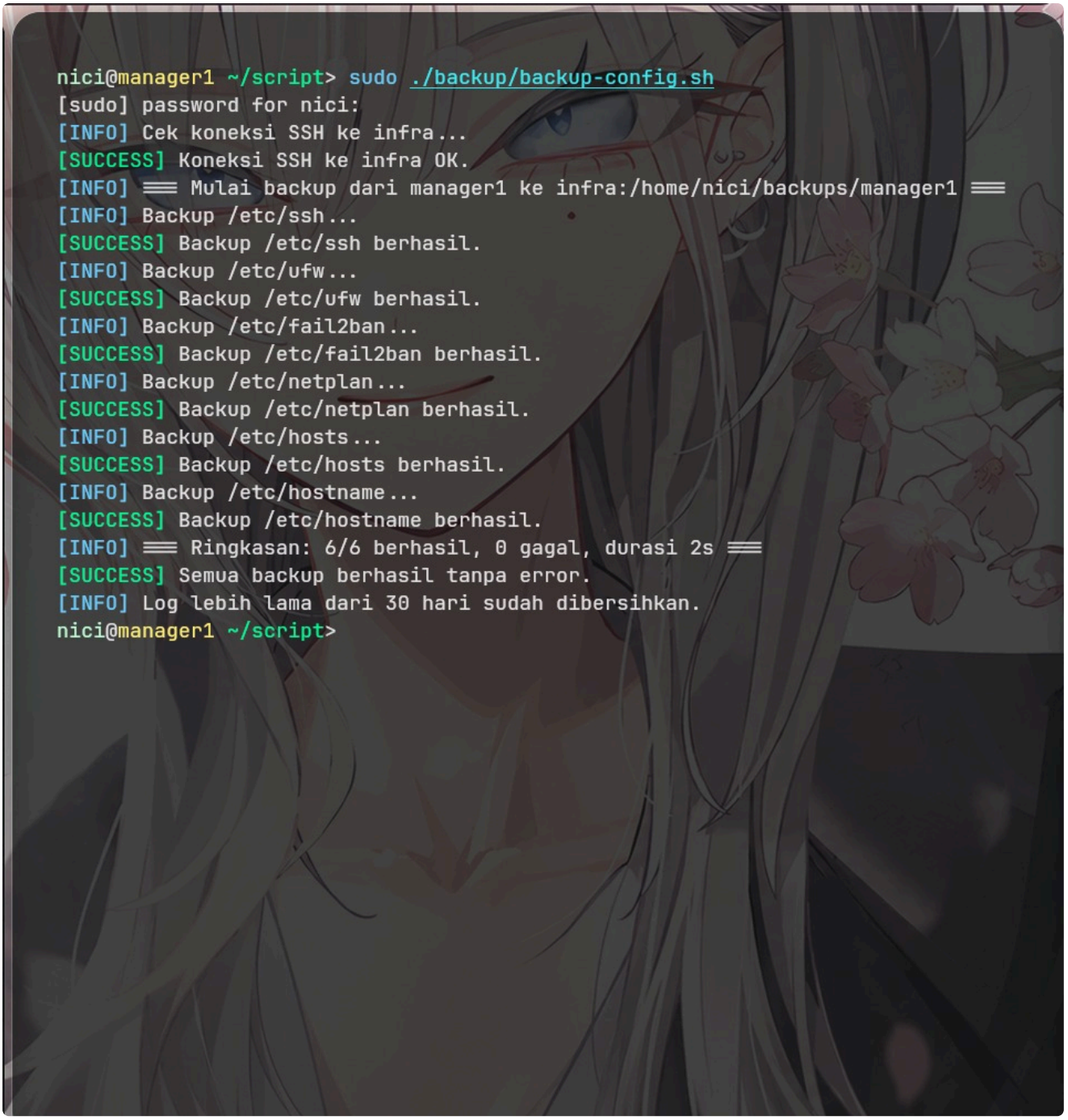
# ==================== MAIN ====================

mkdir -p "$LOG_DIR"
touch "$LOG_FILE"

check_prerequisites
run_backup
result=$?
cleanup_old_logs

exit $result
```

dan kalau kita jalankan kira kira hasilnya seperti ini,



SUID

oke lanjut lagi kita akan masuk ke bagian SUID, disini SUID itu singkatan dari set user id yang mana disini kita akan melakukan audit pada user yang memiliki/meminjam akses root saat menjalankan perintah seperti passwd(karena dasarnya user biasa gabisa ubah password jadi perintah passwd meminjam akses root selama proses berlangsung setelah itu ya udah balik lagi) dan langsung aja kita coba cek file file yang punya SUID aktif dengan command:

```
sudo find / -perm -4000 -type f 2>/dev/null
```

yang hasilnya

```
~/d/devops-btik  (main) ❱ ssh nici@192.168.56.10
nici@manager1 ~> sudo find / -perm -4000 -type f 2>/dev/null
[sudo] password for nici:
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/umount
/usr/bin/su
/usr/bin/passwd
/usr/bin/mount
/usr/bin/sudo
/usr/bin/chsh
/usr/bin/fusermount3
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
```

disitu ada 13 file yang memiliki SUID aktif yang mana jika kita lihat, ke-13 file tersebut itu hanyalah fungsi fungsi normal seperti passwd untuk ganti password, gewgrp,chsh, sudo dll yang mana disini tidak ditemukan indikasi adanya file entah berantah yang memiliki SUID aktif,

auditd

terus masalah audit, kita akan lanjut mencoba untuk melakukan audit dengan menggunakan tools yang bernama auditd, kita langsung aja praktekkan di vm manager1 ku dengan pertama kita harus install dulu dengan command

```
sudo apt install auditd audispd-plugins -y
```

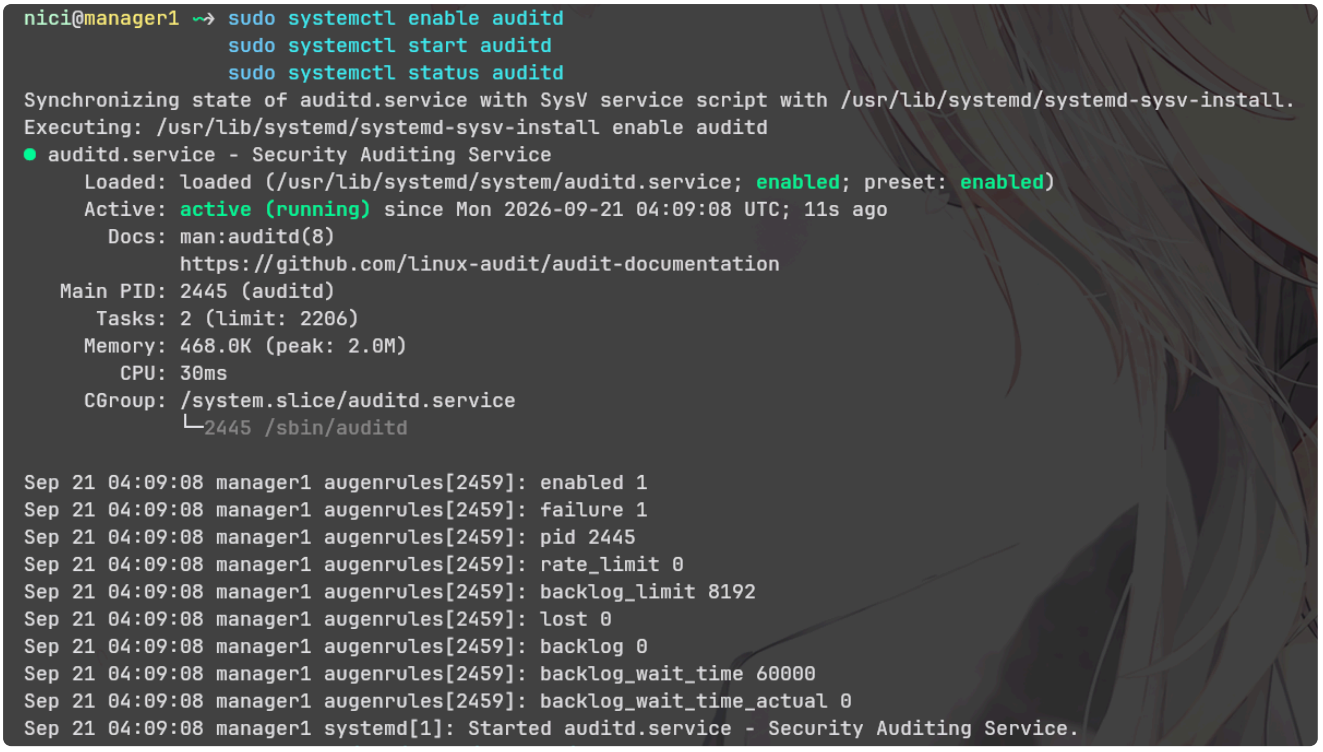
```
nici@manager1 ~> sudo apt install auditd audispd-plugins -y
[sudo] password for nici:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libauparse0t64
The following NEW packages will be installed:
  audispd-plugins auditd libauparse0t64
0 upgraded, 3 newly installed, 0 to remove and 9 not upgraded.
Need to get 312 kB of archives.
```

kira kira begitu hasilnya kalo udah terinstall, lalu kita juga harus untuk enable, start dan sekalian liat status dulu apakah sudah benar benar terinstall atau belum, langsung aja command nya

```
sudo systemctl enable auditd
sudo systemctl start auditd
```

```
sudo systemctl status auditd
```

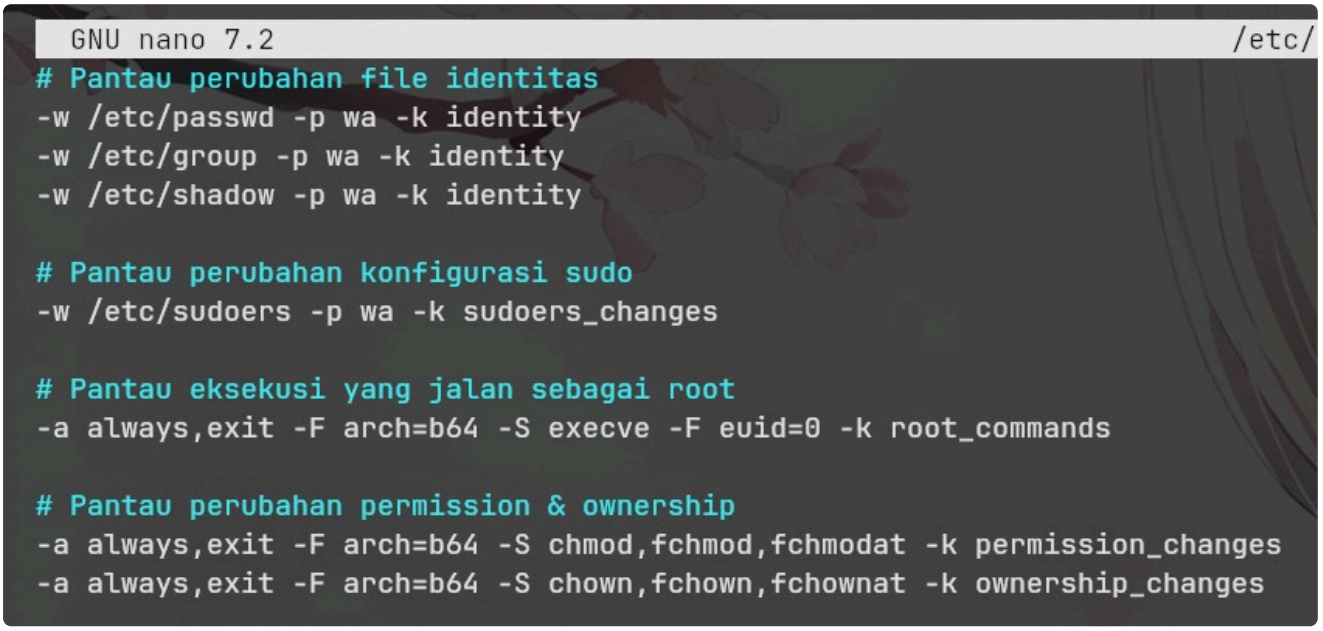
setelah itu hasilnya akan seperti ini,



setelah berjalan, kita langsung aja ketik command

```
sudo nano /etc/audit/rules.d/audit.rules
```

untuk konfigurasi auditd nya aja lalu isi dengan:



konfigurasi tersebut kira kira apa yang kita butuhkan untuk sekarang, lalu setelah itu kita load konfigurasi, restart service dan make sure sekali lagi kalau udah ter konfigurasi atau belum dengan command

```
sudo augenrules --load
sudo systemctl restart auditd
```

cek nya dengan

```
sudo auditctl -l
```

lalu kira kira hasilnya seperti ini

```
nici@manager1 ~> sudo nano /etc/audit/rules.d/audit.rules
nici@manager1 ~> sudo augenrules --load
                 sudo systemctl restart auditd
nici@manager1 ~> sudo auditctl -l
-w /etc/passwd -p wa -k identity
-w /etc/group -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/sudoers -p wa -k sudoers_changes
-a always,exit -F arch=b64 -S execve -F euid=0 -F key=root_commands
-a always,exit -F arch=b64 -S chmod,fchmod,fchmodat -F key=permission_changes
-a always,exit -F arch=b64 -S chown,fchown,fchownat -F key=ownership_changes
```

lalu kita tes ombak aja langsung dengan membuka file konfigurasi di /etc/passwd dan hanya menambahkan spasi saja di bawah sendiri lalu kita lihat apakah ke trigger atau belum

```
sudo nano /etc/passwd
```

setelah nambahin spasi kosongan pada paling bawah sendiri dan di save maka cek apakah auditd nya ketrigger atau tidak dengan command

```
sudo ausearch -k identity -ts recent
```

setelah itu akan keluar kalau kita baru saja ngedit /etc/passwd, hasil gambarnya kaya gini,

```
nici@manager1 ~> sudo ausearch -k identity -ts recent
----
time->Mon Sep 21 04:47:53 2026
type=PROCTITLE msg=audit(1789966073.540:553): proctitle=6E616E6F002F6574632F706173737764
type=PATH msg=audit(1789966073.540:553): item=1 name="/etc/passwd" inode=148167 dev=fc:00 mode=0100644 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_
fe=0 cap_fver=0 cap_frootid=0
type=PATH msg=audit(1789966073.540:553): item=0 name="/etc/" inode=131074 dev=fc:00 mode=040755 ouid=0 ogid=0 rdev=00:00 nametype=PARENT cap_fp=0 cap_fi=0 cap_fe=0 ca
p_fver=0 cap_frootid=0
type=CWD msg=audit(1789966073.540:553): cwd="/home/nici"
type=SYSCALL msg=audit(1789966073.540:553): arch=c000003e syscall=257 success=yes exit=3 a0=ffffff9c a1=5f498094ef50 a2=241 a3=1b6 items=2 ppid=2971 pid=2972 auid=100
0 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts1 ses=13 comm="nano" exe="/usr/bin/nano" subj=unconfined key="identity"
nici@manager1 ~>
```

Gitleaks

cukup segitu buat auditd lalu kita langsung masuk ke gitleaks, intinya kita pake gitleaks untuk cek apakah ada sesuatu yang terbocorkan pada repository kita dan langsung saja kita install dengan command:

```
sudo pacman -S pre-commit
```

install dulu itu pre-commit nya boskyuh lalu gitleaks nya, karena di arch gaada ya kita tembak link repo githubnya aja dengan:

```
# 1. Install pre-commit
sudo pacman -S pre-commit
pre-commit --version

# 2. Install manual dari GitHub release karena tidak ada di repo arch
curl -sSL
https://github.com/gitleaks/gitleaks/releases/download/v8.30.1/gitleaks_8.
30.1_linux_x64.tar.gz -o gitleaks.tar.gz
tar -xzf gitleaks.tar.gz gitleaks
mkdir -p ~/.local/bin
mv gitleaks ~/.local/bin/
chmod +x ~/.local/bin/gitleaks
rm gitleaks.tar.gz
fish_add_path ~/.local/bin
gitleaks version

# 3. Bikin config di root repo
cd ~/d/devops-btik
nano .pre-commit-config.yaml
```

nah pada konfigurasi pre commit, kita isi dengan ini:



lalu setelah itu kita install hook nya dengan command:

```
pre-commit install
```

dan kita akan tes dengan secret random aja pakai token random dengan command

```
echo "GITHUB_TOKEN=ghp_$(openssl rand -hex 20)" > ohyess.txt
```

lalu jangan lupa kita commit ygy dan harusnya hasilnya seperti ini:

```
~/d/devops-btik  (main) ❱ git commit -m "coba pake key aneh ohyes"
Detect hardcoded secrets.....................................................Failed
- hook id: gitleaks
- exit code: 1

○
    │╲
    │ ○
    ○ ▒
    ▒    gitleaks

Finding:     GITHUB_TOKEN=REDACTED
Secret:      REDACTED
RuleID:      generic-api-key
Entropy:     4.018884
File:        ohyess.txt
Line:        1
Fingerprint: ohyess.txt:generic-api-key:1

Finding:     GITHUB_TOKEN=REDACTEDb228
Secret:      REDACTED
RuleID:      github-pat
Entropy:     4.006198
File:        ohyess.txt
Line:        1
Fingerprint: ohyess.txt:github-pat:1

2:17PM INF 1 commits scanned.
2:17PM INF scan completed in 3.99ms
2:17PM WRN leaks found: 2
```

lalu kita lanjut dengan pre-commit oh ya sebelum masuk dari situ untuk ohyess.txt nya kita hapus ya dan cek apakah masih ada staged file pake git status yang harusnya seperti ini hasilnya

```
~/d/devops-btik  (main) ❱ rm ohyess.txt
~/d/devops-btik  (main) ❱ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   ohyess.txt

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    ohyess.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        script/

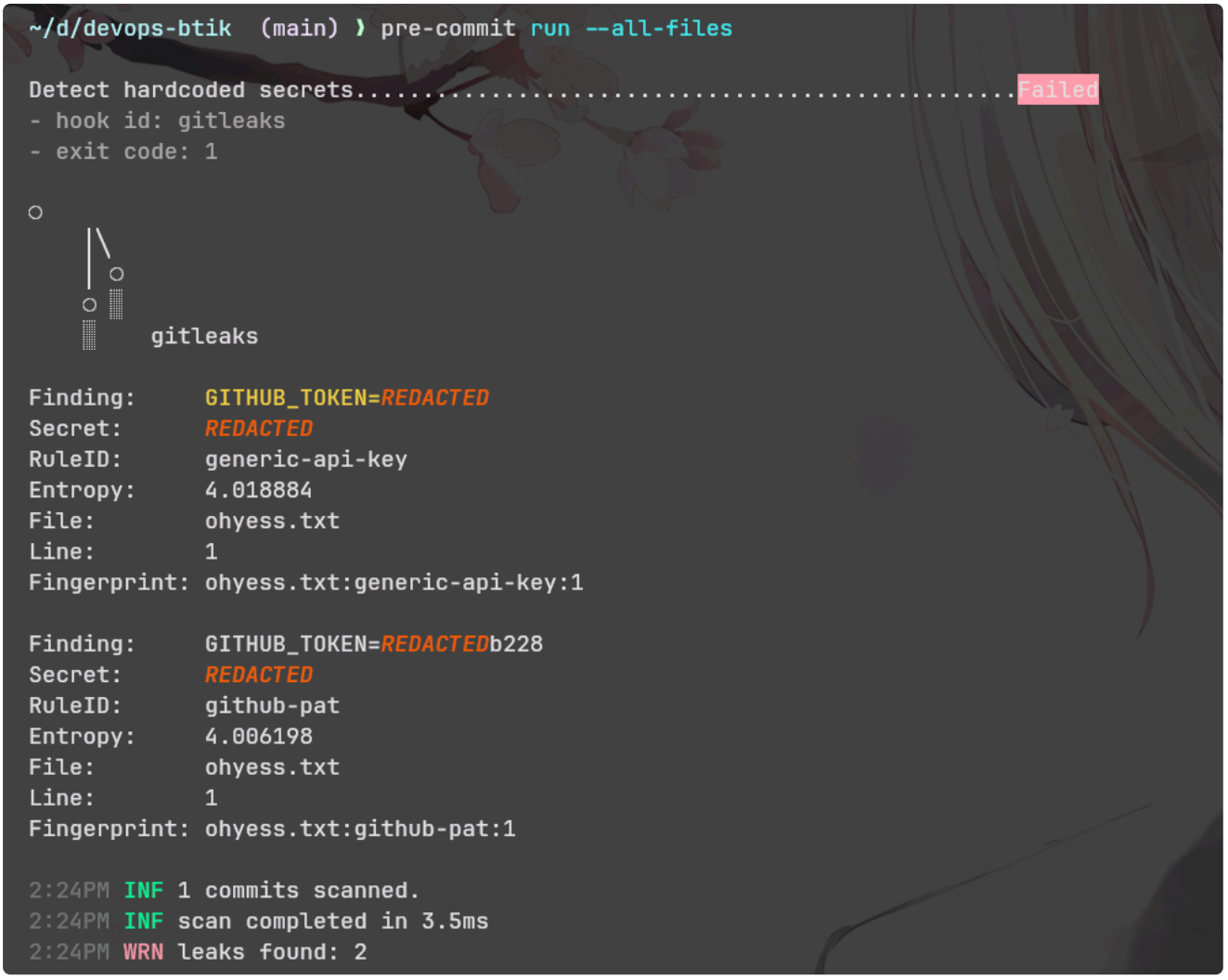
~/d/devops-btik  (main) ❱
```

lalu kita lanjut dengan pre-commit

disini kita langsung aja bakalan scan semua file yang ada di repo dengan command

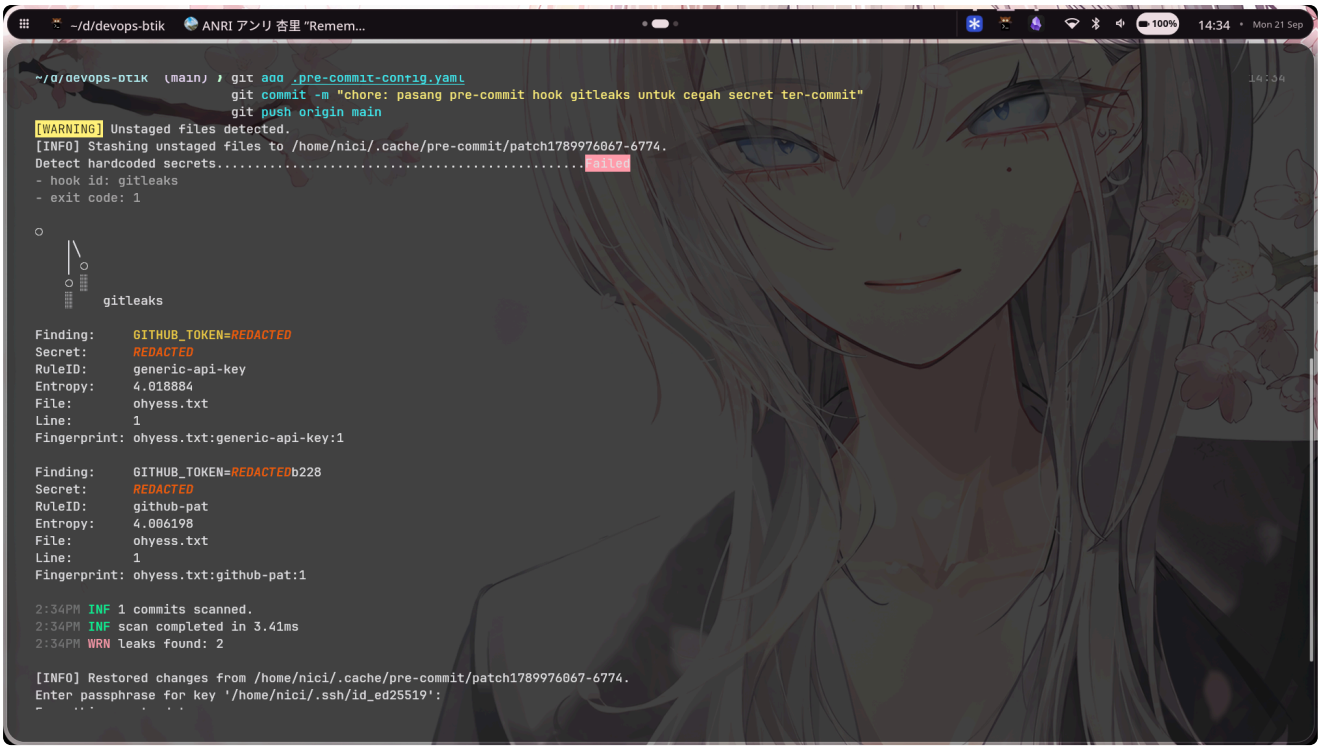
```
pre-commit run --all-files
```

yang hasilnya seperti ini,



lalu kita akan commit saja pre-commit nya dan sekalian di push saja dengan command

```
git add .pre-commit-config.yaml
git commit -m "chore: pasang pre-commit hook gitleaks untuk cegah secret
ter-commit"
git push origin main
```



dan disitu kita udah ter push, gitu aja dokumentasi kali ini yang menghabiskan 90 persen dari 100 persen waktu ngoprek ku